Viajaí Bolso

Documentação Técnica e Funcional do Projeto

App de controle de gastos em viagens — Flutter

Projeto desenvolvido como trabalho de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas, em fase de evolução para se tornar um aplicativo funcional completo.

Documento gerado e atualizado automaticamente a cada alteração relevante no código.

1. Visão Geral do Projeto

O **Viajaí Bolso** é um aplicativo mobile (Flutter) para controle financeiro de viagens: o usuário cadastra viagens — cada uma com destino, período e orçamento — e registra despesas associadas a cada viagem, categorizadas (alimentação, transporte, etc.), acompanhando o quanto já foi gasto em relação ao orçamento definido.

O projeto nasceu como protótipo de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas e agora está em evolução para se tornar um aplicativo funcional completo, a ser apresentado como projeto final do curso. Este documento acompanha essa evolução: a cada mudança relevante no código, uma nova seção é adicionada ao **Histórico de Alterações** (seção 7), mantendo sempre a descrição das seções anteriores atualizada com o estado mais recente do app.

1.1 Principais dependências (pubspec.yaml)

Pacote	Finalidade
supabase_flutter	Cliente oficial do Supabase: autenticação, banco de dados (Postgres) e armazenamento de arquivos
image_picker / image_picker_for_web	Seleção de fotos (perfil, viagem e comprovante de despesa), enviadas ao Supabase Storage
intl	Suporte a internacionalização/formatação (incluído, uso pontual)
http (v4)	Consulta à API pública de câmbio (open.er-api.com) usada na conversão de moeda da tela inicial

1.2 Arquitetura em camadas

O código em `lib/` está organizado em três camadas, sem uso de gerenciador de estado externo (Provider, Riverpod, Bloc) — o controle de estado é feito manualmente com `setState`:

- **constants.dart** — configurações globais: URL/chave do projeto Supabase e paleta de cores do app.
- **models/** — classes de dados puras (Despesa, Categoria, Usuario, Viagem), cada uma com método `fromJson` para converter a resposta do banco.
- **services/** — camada de acesso ao Supabase (AuthService, ViagemService, DespesaService, DashboardService, StorageService) e à API de câmbio (CurrencyService, v4), usando `supabase_flutter` e `http`. O ChatService (v5) também entra aqui, mas não chama nenhuma API de IA diretamente — ele aciona uma Edge Function do Supabase, que é quem de fato fala com o provedor de IA (ver seção 3.4).
- **utils/** (v4) — funções auxiliares puras, sem estado: lista de moedas e formatação de valores (moedas.dart) e geração do CSV de exportação de despesas (exportacao.dart).
- **screens/** — telas do app, que chamam os services diretamente (não há camada intermediária de controller/repository).

Atualização (v2): o app não depende mais de um servidor próprio — desde a migração para o Supabase (seção 7, versão 2), o backend é hospedado, acessado sempre via HTTPS, com a segurança de cada usuário só acessar os próprios dados garantida por políticas de Row Level Security (RLS) diretamente no banco (ver `supabase/schema.sql`).

1.3 Acesso direto ao app (versão web)

Atualização (v3): a cada atualização enviada ao repositório, o GitHub Actions gera automaticamente a versão web do app e publica no GitHub Pages, gerando um link direto para acesso pelo navegador, sem necessidade de instalar nada — útil para demonstração na banca (ver seção 7, versão 3). **Atualização**

(v11): o repositório foi renomeado para que o link ficasse mais curto, sem o sufixo com o nome do projeto (ver seção 7, versão 11):

- `https://viajaaibolso-design.github.io/`

2. Modelos de Dados (lib/models)

2.1 Despesa e Categoria — despesa.dart

Representa uma despesa individual vinculada a uma viagem e a uma categoria.

Campo (Despesa)	Tipo	Descrição
idDespesa	String?	Identificador único (uuid gerado pelo Supabase)
descricao	String	Descrição da despesa
valor	double	Valor gasto
data / hora	String / String?	Data e hora do gasto
formaPagamento	String	Ex.: Cartão de crédito, Débito, Dinheiro, Pix, Outros
idViagem / idCategoria	String	Referências (uuid) à viagem e categoria associadas
categoria / icone	String? / String?	Nome e ícone da categoria (via join com categorias)
fotoUrl (v4)	String?	URL do comprovante anexado (Supabase Storage), RF22

A classe **Categoria** representa uma categoria de despesa (id, nome, ícone), lida do banco.

2.2 Usuario — usuario.dart

Campo	Tipo	Descrição
idUsuario	String	Identificador do usuário (uuid do Supabase Auth)
nome / email	String	Dados básicos de cadastro
fotoUrl	String?	URL pública da foto de perfil no Supabase Storage
moedaPadrao	String	Moeda padrão do usuário (default: 'BRL') — RF29, também usada na conversão da home (RF52/RF53)
viagemAtivaId (v4)	String?	Viagem escolhida manualmente como ativa no dashboard (RF14); null = automático

2.3 Viagem — viagem.dart

Campo	Tipo	Descrição
idViagem	String?	Identificador da viagem (uuid)
nome / destino	String	Nome e destino da viagem
fotoUrl	String?	URL pública da foto da viagem no Supabase Storage
dataInicio / dataFim	String	Período da viagem (AAAA-MM-DD)
orcamento	double	Orçamento definido para a viagem
totalGasto / percentualGasto	double	Valores agregados, calculados pela view viagens_resumo no banco
idUsuario	String?	Dono da viagem

Campo	Tipo	Descrição
moedaLocal (v4)	String	Moeda do destino (default: 'BRL') — RF52, definida ao cadastrar a viagem

2.4 MensagemChat (v5) — mensagem_chat.dart

Representa uma mensagem do assistente de IA (v5 — ver nota sobre a renumeração desse recurso na seção 6), seja do usuário ou da resposta gerada pelo Gemini. Persistida na tabela `mensagens` do Supabase.

Campo	Tipo	Descrição
conversaId	String	Conversa à qual a mensagem pertence (tabela conversas)
papel	String	'user' (pergunta do usuário) ou 'assistant' (resposta da IA)
conteudo	String	Texto da mensagem
criadoEm	DateTime?	Data/hora de envio

3. Serviços — Integração com o Supabase (lib/services)

Desde a versão 2 (seção 7), os serviços não fazem mais requisições HTTP a um servidor próprio: eles usam o cliente `supabase_flutter` para falar direto com a autenticação, o banco Postgres e o armazenamento de arquivos do projeto Supabase. A segurança — cada usuário só acessa os próprios dados — é garantida por políticas de Row Level Security (RLS) no banco, não por checagens manuais no app (ver `supabase/schema.sql`).

3.1 AuthService — autenticação e perfil

Método	Recurso do Supabase	Função
cadastrar(nome, email, senha)	auth.signUp	Cria uma nova conta (dispara e-mail de confirmação)
login(email, senha)	auth.signInWithPassword	Autentica o usuário
logoff()	auth.signOut	Encerra a sessão (local e no Supabase)
recuperarSenha(email)	auth.resetPasswordForEmail	Envia código de recuperação por e-mail
redefinirSenha(...)	auth.verifyOTP + auth.updateUser	Valida o código e define a nova senha
getPerfil(idUsuario)	tabela profiles	Busca os dados do perfil
atualizarPerfil(...)	tabela profiles + auth.updateUser	Atualiza nome, e-mail, moeda e/ou foto
alterarSenha(...)	auth.signInWithPassword + auth.updateUser	Reautentica e troca a senha
uploadFotoPerfil(...)	Storage (bucket fotos)	Envia a foto de perfil e devolve a URL pública

3.2 ViagemService, DespesaService e DashboardService

Classe / Método	Recurso do Supabase	Função
ViagemService.listar(idUsuario)	view viagens_resumo	Lista as viagens do usuário (já com totais calculados)
ViagemService.cadastrar / atualizar / remover	tabela viagens	CRUD de viagens
ViagemService.definirViagemAtiva(...) (v4)	tabela profiles	RF14 — grava a viagem escolhida como ativa
DespesaService.listar(idViagem)	tabela despesas (+ join categorias)	Lista despesas de uma viagem
DespesaService.listarCategorias()	tabela categorias	Lista categorias disponíveis
DespesaService.cadastrar / atualizar / remover	tabela despesas	CRUD de despesas (agora inclui foto_url)

Classe / Método	Recurso do Supabase	Função
DashboardService.getDashboard(idUsuario)	consultas combinadas	Monta viagem ativa (manual ou automática, v4), gastos de hoje, maior gasto e últimas despesas; as 5 consultas rodam em paralelo com Future.wait em vez de uma atrás da outra (v13)
StorageService.upload(...)	Storage (bucket fotos)	Upload genérico (fotos de perfil/viagem, comprovantes e exportações CSV)

Observação técnica: a checagem de dono dos dados (usuário só vê as próprias viagens/despesas) agora é feita pelo banco via RLS, não pelo código do app — isso reduz o risco de um bug no client expor dados de outro usuário. Erros ainda são tratados de forma genérica nas telas (mensagem fixa "Erro de conexão com o servidor"); mensagens mais específicas por tipo de erro continuam sendo um ponto de melhoria.

3.3 CurrencyService (v4) — conversão de câmbio

Novo serviço que consulta a API pública e gratuita `open.er-api.com` (sem necessidade de chave) para converter o total gasto da viagem (na moeda local) para a moeda padrão do usuário, usada na tela inicial (RF52/RF53). As cotações ficam em cache na memória do app por 6 horas, guardando também o horário da última consulta para exibir ao usuário (RNF23). Se a API estiver indisponível, a tela inicial mostra "Conversão indisponível no momento" em vez de travar ou quebrar (RNF17).

3.4 ChatService (v5) — assistente de IA

Serviço do assistente de viagem por IA (v5 — ver nota sobre a renumeração desse recurso na seção 6). É importante destacar uma decisão de arquitetura: o **app nunca fala diretamente com a API de IA**. A chave dessa API é um segredo de verdade — diferente da anonKey do Supabase, que é pública por design e só é segura porque toda consulta passa pelas políticas de RLS do banco. Se essa chave fosse colocada no código do app, ela ficaria exposta por completo na versão web (qualquer pessoa consegue abrir o DevTools do navegador e ler o código gerado). Por isso, entre o app e o provedor de IA existe uma **Edge Function do Supabase** (`supabase/functions/chat-ia`), que roda no servidor e é a única parte do sistema que conhece a chave — guardada como "secret" do projeto, nunca commitada no Git.

Provedor de IA: Gemini (Google AI Studio). O plano inicial era usar a API da Anthropic (Claude), mas ela exige cartão de crédito já no cadastro — mesmo para liberar o crédito gratuito de teste. Como o Google AI Studio oferece uma chave de API gratuita sem exigir cartão, o provedor foi trocado. A arquitetura (chave só no servidor, dentro da Edge Function) não muda com o provedor escolhido; se um dia for preciso trocar de novo, apenas o arquivo da Edge Function precisa mudar.

Método / Componente	Recurso	Função
ChatService.obterOuCriarConversa()	tabela conversas	Recupera a conversa mais recente do usuário ou cria uma nova
ChatService.listarMensagens(...)	tabela mensagens	Carrega o histórico salvo ao abrir a tela
ChatService.enviarMensagem(...)	tabela mensagens + Edge Function chat-ia (modo chat)	Salva a pergunta, aciona a Edge Function com o histórico e o contexto da viagem ativa, e salva a resposta. Timeout de 45s (v14).

Método / Componente	Recurso	Função
ChatService.planejarViagem(...) (v9)	tabela mensagens + Edge Function chat-ia (modo roteiro)	Salva o resumo da entrevista de planejamento como pergunta, aciona a Edge Function pedindo um roteiro, e salva a resposta. Timeout de 60s (v14).
Edge Function chat-ia (Deno)	API do Gemini (Google AI Studio)	Único ponto do sistema que guarda a chave da API; monta o prompt (modo chat ou roteiro) e devolve a resposta gerada

No modo chat (padrão), o prompt enviado ao Gemini inclui um resumo da viagem ativa do usuário (nome, destino, orçamento, total gasto, percentual do orçamento e últimas despesas), permitindo respostas contextualizadas (ex.: "quanto já gastei hoje?"). Apenas usuários autenticados conseguem acionar a função — comportamento padrão do Supabase para Edge Functions, sem necessidade de código extra.

Modo roteiro (v9): quando o usuário preenche a entrevista de planejamento (destino, datas, orçamento, meio de transporte e preferências) na tela do Agente de vIAgens, a mesma Edge Function chat-ia é chamada com `modo:'roteiro'` e esses dados, mas com um prompt diferente: pede ao Gemini um roteiro dia a dia, estimativa de custos por categoria, sugestões de passagem/hospedagem/restaurante (faixas de preço aproximadas, nunca marcas ou preços reais) e, se o meio de transporte for carro, dicas de estrada (trecho, pedágio estimado, pontos de parada, documentação do veículo). O prompt instrui o modelo a deixar claro que tudo é estimativa — não há consulta a nenhuma API de passagem/hospedagem real, por decisão do usuário (ver seção 6). A resposta entra no histórico da conversa como uma mensagem do assistente, igual a qualquer resposta do modo chat.

4. Telas e Funcionalidades (lib/screens)

SplashScreen

Tela inicial. Exibe a logo por 2 segundos e verifica se há uma sessão ativa no Supabase Auth. Se houver, leva direto ao MainScreen; senão, à tela de Login.

LoginScreen

Login com e-mail e senha, opção de mostrar/ocultar senha, links para "Esqueci minha senha" e "Cadastre-se". Em caso de sucesso, o Supabase Auth já mantém a sessão salva automaticamente e o app abre o MainScreen.

CadastroScreen

Cadastro de novo usuário (nome, e-mail, senha, confirmação de senha) com aceite obrigatório dos Termos de Uso — o texto "Termos de Uso e Política de Privacidade" agora é um link que abre a TermosScreen (v4).

RecuperarSenhaScreen

Fluxo de recuperação de senha em duas etapas: solicitar código por e-mail e, em seguida, informar o código recebido junto com a nova senha.

MainScreen

Shell de navegação principal, com barra inferior de 5 abas: Início (Dashboard), Viagens, Despesas, Agente (v5/v6) e Perfil.

DashboardScreen

Tela inicial pós-login: saudação personalizada, resumo da viagem ativa — manual ou automática (RF14, v4) — na moeda local da viagem, com indicador circular de progresso (fica terracota quando o orçamento estoura); tocar no indicador alterna entre mostrar o percentual gasto e o saldo real restante (orçamento menos total gasto, já formatado na moeda da viagem), com um rótulo "toque p/ saldo"/"toque p/ %" indicando a ação disponível (v12). Mostra também valor convertido para a moeda padrão do usuário com seletor de moeda e horário da cotação (RF52/RF53, v4), gastos do dia, maior gasto e lista das despesas mais recentes com botão "Ver todas" agora funcional (RF28, v4). Suporta pull-to-refresh.

ViagensScreen

Lista todas as viagens cadastradas em cards (foto, nome, destino, datas, progresso do orçamento), com opções de editar e excluir cada viagem, marcar/desmarcar como viagem ativa do dashboard (ícone de estrela, RF14, v4), e botão para cadastrar uma nova.

NovaViagemScreen

Formulário de criação/edição de viagem: foto (câmera/galeria), nome, destino, datas de início e fim, orçamento e moeda local do destino (RF52, v4).

DespesasScreen

Seleciona uma viagem e lista suas despesas, com filtro por categoria. O cadastro de despesa (painel deslizante) é sempre preenchido manualmente, com um anexo de foto do comprovante simples e opcional (RF22, v4/v8) — sem nenhuma leitura automática por IA; a leitura automática de nota fiscal por foto (v6) e por QR Code (v7) foi removida na v8, por decisão do usuário. Permite editar e excluir despesas pelo mesmo formulário. Mostra o total das despesas filtradas numa barra fixa na parte inferior e permite exportá-las em CSV (RF23, v4, via link do Supabase Storage). O botão de nova despesa ("+") fica como ícone discreto no canto superior direito da barra de título, ao lado do botão de exportar (v8) — antes era um botão flutuante que sobrepunha a barra do Total.

PerfilScreen
Exibe e permite editar os dados do usuário (foto, nome, e-mail), alterar a senha, escolher a moeda padrão (RF29, v4) e encerrar a sessão (logoff). Dá acesso às novas telas de Configurações, Sobre o app e Suporte (RF30/RF31/RF32, v4).

ConfiguracoesScreen (v4)
Preferência de notificações e atalho para os Termos de Uso (RF30).

SobreScreen (v4)
Informações sobre o app (versão, descrição, contexto do TCC) e atalho para os Termos de Uso (RF31).

SuporteScreen (v4)
Canal de contato (e-mail copiável) e perguntas frequentes (RF32).

TermosScreen (v4)
Conteúdo real dos Termos de Uso e Política de Privacidade, atendendo à LGPD (RF07/RNF14) — antes era só um texto estático sem tela própria.

ChatScreen (v5/v6/v9)
Tela do Agente de vIAgens — renomeada na v6 de "Assistente de IA" para "Agente de vIAgens", já alinhada com a terminologia do levantamento de requisitos. Tem dois modos: o chat livre de perguntas e respostas sobre a viagem ativa (v5), e, desde a v9, o botão "Planejar viagem com IA" (logo abaixo da barra de título), que abre um formulário de entrevista (destino, datas, orçamento, meio de transporte, preferências) e devolve, como uma mensagem do agente no próprio chat, um roteiro dia a dia com estimativa de custos e sugestões de passagem/hospedagem/restaurante — e dicas de estrada, se o meio de transporte for carro. Balões de mensagem (usuário à direita, agente à esquerda), indicador de "digitando..." enquanto aguarda a resposta, e histórico persistido no banco (recarregado toda vez que a tela é aberta). Ainda não é o Agente de vIAgens completo (RF33–RF47) — documentação/vistos/vacinas e integração com APIs reais de passagem/hospedagem seguem fora do escopo (ver seção 6).

5. Fluxo de Navegação

Resumo do fluxo entre as telas do aplicativo, a partir da inicialização:

- **Splash** → verifica sessão salva → **Login** (se não houver sessão) ou **MainScreen** (se já houver uma sessão salva).
- **Login** → sucesso leva ao **MainScreen**; "Cadastre-se" leva ao **Cadastro**; "Esqueci minha senha" leva à **Recuperação de Senha**.
- **MainScreen** organiza 5 abas fixas: **Dashboard** | **Viagens** (→ Nova/Editar Viagem) | **Despesas** (formulário em painel deslizante) | **Agente** (v5, chat de IA, renomeado na v6 de "Assistente") | **Perfil**.
- **Perfil** → **Sair da conta** encerra a sessão e retorna à tela de **Login**, limpando todo o histórico de navegação.

6. Estado Atual e Pontos de Evolução

Como protótipo acadêmico, o app já demonstra o fluxo completo de cadastro, login, gestão de viagens e despesas. Os pontos abaixo estão mapeados como próximos passos para transformá-lo em um app funcional completo — úteis tanto para o planejamento das próximas entregas quanto para a banca de apresentação:

Nota sobre a renumeração dos requisitos do Agente de IA: o levantamento de requisitos foi atualizado entre a v5 e a v6 deste documento, e a numeração RF33–RF40 (usada na versão 5 deste documento) mudou de significado. No levantamento atual, RF33–RF47 correspondem ao **Agente de vIAgens** — um consultor de viagens completo (entrevista prévia, roteiro com gastos estimados, indicação de passagens/hospedagem/restaurantes, apoio a trajetos de carro e orientação sobre documentação e vacinas). Na v9, o usuário decidiu avançar parte desse bloco sem esperar a validação completa com o orientador: entrevista prévia, roteiro com gastos estimados, sugestões de passagem/hospedagem/restaurante e apoio a viagens de carro já estão implementados (seção 3.4), sempre como estimativa da própria IA — a decisão entre usar só a IA ou integrar APIs reais de viagem (preços reais de passagem/hospedagem) foi resolvida a favor de usar só a IA, por ser mais simples e gratuito para o escopo do TCC. Documentação/vistos/vacinas ainda não foi implementada e continua dependendo de validação com o orientador, assim como qualquer evolução para dados reais de preço.

- **Resolvido na v2:** o backend deixou de ser um servidor fixo em IP local — agora é o Supabase, acessado via HTTPS, com autenticação e regras de acesso reais (RLS).
- **Resolvido na v3:** o app passou a ter um link direto e público, gerado automaticamente a cada atualização do código (GitHub Actions + GitHub Pages) — não é mais necessário instalar nada para demonstrá-lo. Primeiro deploy já confirmado funcionando.
- **Resolvido na v4:** nova paleta de cores, câmbio na tela inicial, seleção manual de viagem ativa, seletor de moeda padrão, telas de Configurações/Sobre/Suporte, Termos de Uso com conteúdo real, anexo de comprovante e exportação de despesas em CSV — ver seção 7, versão 4.
- **Resolvido na v5:** assistente de viagem por IA (chat simples, ver nota de renumeração acima), com a chave da API do Gemini protegida no servidor (Edge Function do Supabase) em vez de embutida no app — ver seção 7, versão 5.
- **Resolvido na v6:** leitura automática de notas fiscais por visão computacional (RF48–RF51), reaproveitando a mesma Edge Function/chave de IA do assistente — ver seção 7, versão 6.
- **Resolvido na v7:** segundo caminho de leitura de nota fiscal, agora pelo QR Code da NFC-e (decodificado localmente no aparelho, sem IA nesse passo), com uma nova Edge Function que consulta a nota direto no site da Receita e usa o Gemini só para ler o texto da página — cobertura inicial limitada a notas da Paraíba (PB), com fallback automático para o fluxo por foto/manual nos demais casos — ver seção 7, versão 7.
- **Resolvido na v8:** removida a leitura automática de nota fiscal (os dois caminhos, por foto da v6 e por QR Code da v7) — depois de ver a função funcionando de verdade na tela de despesas, o usuário decidiu simplificar o fluxo de cadastro: agora é sempre manual, com um anexo de foto do comprovante simples e opcional (RF22), sem nenhuma chamada de IA nesse passo. Também foi corrigido o botão "+" de nova despesa, que como FloatingActionButton ficava sobrepondo o valor do Total na parte inferior da tela — ver seção 7, versão 8.
- **Resolvido na v9:** parte do Agente de vIAgens completo (RF33–RF47) avançada sem esperar a validação formal com o orientador, por decisão do usuário: entrevista prévia de planejamento, roteiro dia a dia com estimativa de custos, sugestões de passagem/hospedagem/restaurante e apoio a viagens de carro — tudo reaproveitando a mesma Edge Function/chave de IA do chat (modo "roteiro" em vez de "chat"), sempre como estimativa da IA, sem integrar nenhuma API real de preços — ver seção 7, versão 9.
- **Resolvido na v10:** corrigido um bug real na exportação de despesas (RF23) — o CSV era montado com `codeUnits` em vez de UTF-8 de verdade, o que corrompia qualquer caractere fora do alfabeto latino básico (ex.: emoji numa descrição de despesa) na hora de virar bytes para o upload. Passou a usar `utf8.encode` com um BOM no início do arquivo, pra também evitar acentos errados ao abrir no Excel do

Windows. A mensagem de erro da exportação, que antes era sempre genérica ("Erro ao gerar exportação", sem detalhe nenhum), agora mostra o motivo real devolvido pelo Supabase — ver seção 7, versão 10.

- **Resolvido na v11:** o link de acesso ao app (seção 1.3) ficou mais curto — o repositório no GitHub foi renomeado para `viajaaibolso-design.github.io`, o que faz o GitHub Pages servir o site direto na raiz, sem o sufixo `/viaja-ai-bolso/`. Mudança restrita à infraestrutura de deploy, sem alteração de código Dart, banco ou Edge Functions — ver seção 7, versão 11.
- **Pendente de validação com o orientador:** dentro do Agente de vIAgens completo (RF33–RF47), só falta a orientação sobre documentação e vacinas — ainda depende de validação com o orientador (e, se possível, com usuários reais) antes de ser implementada, conforme o próprio levantamento de requisitos pede. Uma eventual evolução para preços reais de passagem/hospedagem (em vez de estimativa da IA) também dependeria dessa validação, por envolver contratar uma API paga.
- **Pendente do lado do usuário:** o assistente de IA (v5) só funciona de verdade depois que a Edge Function chat-ia for publicada no Supabase e o secret GEMINI_API_KEY estiver configurado com uma chave criada em aistudio.google.com — sem isso, a tela abre normalmente, mas a IA não responde. As Edge Functions extrair-nota e extrair-nota-qrcode (v6/v7), da função de leitura de nota fiscal removida na v8, deixaram de ser chamadas pelo app, mas continuam publicadas no painel do Supabase até serem excluídas manualmente por ali, se o usuário quiser liberar esse espaço.
- **Parcialmente testado:** o app já foi executado de verdade pela primeira vez fora deste ambiente — cadastro de usuário funcionou e o e-mail de confirmação do Supabase chegou normalmente. Login, upload de foto, CRUD completo de viagens/despesas, as novidades da v4 (câmbio, viagem ativa, comprovante, exportação), o assistente de IA (v5) e o modo roteiro do Agente de vIAgens (v9) ainda não foram confirmados em uso real.
- Por padrão, o Supabase exige confirmação de e-mail para novas contas — vale revisar essa configuração no painel do projeto (Authentication) para decidir se isso é desejável na demonstração.
- Tratamento de erros de rede ainda é genérico (mensagem fixa) na maior parte das telas, sem diferenciar tipos de erro — exceções feitas à conversão de câmbio, que já trata indisponibilidade da API separadamente (RNF17), e à exportação de despesas (RF23, v10), que agora mostra o erro real devolvido pelo Supabase em vez de uma mensagem fixa.
- RF23 (exportação) foi implementada em CSV, não em PDF — suficiente para abrir em Excel/Sheets, mas vale registrar essa escolha caso a banca pergunte especificamente por PDF.
- Não há persistência local (offline) dos dados de viagens/despesas — o app depende de conexão com o Supabase.
- Não há rotas nomeadas nem testes automatizados no projeto até o momento.

7. Histórico de Alterações

Esta seção registra, em ordem cronológica, as mudanças feitas no projeto a partir desta documentação — servindo como registro de evolução para a apresentação final.

Versão 1 — Estado inicial do protótipo (baseline)

Data: 27/08/2026

Documentação inicial gerada a partir do código-fonte existente do projeto (entregue ao final do 3º período), antes do início de novas implementações. Estabelece o ponto de partida do histórico de mudanças que será usado para a apresentação final do curso.

- Levantamento completo da arquitetura atual (models, services, screens)
- Nenhuma alteração de código foi feita nesta etapa — apenas documentação do estado existente

Versão 2 — Migração do backend para o Supabase

Data: 03/09/2026

O backend próprio do protótipo (servidor fixo em 192.168.0.9:5000) foi substituído pelo Supabase: banco de dados Postgres hospedado, autenticação (login, cadastro, recuperação de senha) e armazenamento de arquivos (fotos de perfil e de viagens), tudo acessado via HTTPS. O app Flutter agora fala direto com o Supabase pelo pacote oficial supabase_flutter, sem depender de nenhum servidor próprio rodando. Também foi configurado um repositório Git local no projeto para acompanhamento das mudanças pelo VSCode.

- Criado o schema do banco (supabase/schema.sql): tabelas profiles, categorias, viagens e despesas, com segurança por usuário via Row Level Security (RLS) e bucket de Storage para fotos
- AuthService reescrito para usar Supabase Auth (login, cadastro, logoff, recuperação de senha por código, alteração de senha e perfil)
- ViagemService, DespesaService e DashboardService reescritos para consultar o Postgres do Supabase diretamente, respeitando as políticas de RLS
- Fotos de perfil e de viagens passam a ser enviadas para o Supabase Storage (antes eram salvas como texto base64 dentro dos próprios registros)
- Identificadores (usuário, viagem, despesa, categoria) passam de números inteiros para UUID, seguindo o padrão do Supabase
- Removida a dependência de SharedPreferences para controlar a sessão — o Supabase Auth já mantém isso automaticamente
- Configurado repositório Git local no projeto (commits com histórico de mudanças, visível pelo VSCode)
- Ainda não testado em um app rodando de verdade (não foi possível compilar o Flutter neste ambiente) — recomenda-se rodar o app e testar cadastro/login/CRUD antes da apresentação

Versão 3 — Publicação da versão web (link direto de acesso)

Data: 09/09/2026

Foi criado um fluxo automático (GitHub Actions) que gera a versão web do app a cada atualização enviada ao repositório e publica o resultado no GitHub Pages. Com isso, o projeto passa a ter um link direto, acessível por qualquer navegador, sem precisar instalar nada — ideal para demonstrar o app durante a apresentação do TCC.

- Criado o workflow .github/workflows/deploy.yml, que instala o Flutter, roda 'flutter pub get' e 'flutter build web' automaticamente a cada push na branch main
- Build da web configurado com --base-href /viaja-ai-bolso/ para funcionar corretamente no endereço do GitHub Pages
- Publicação automática do resultado do build (pasta build/web) no GitHub Pages via actions/deploy-pages
- Necessário habilitar manualmente 'GitHub Actions' como origem em Settings > Pages do repositório (passo único, feito pela interface do GitHub)
- Link de acesso ao app: https://viajaaibolso-design.github.io/viaja-ai-bolso/
- Primeiro deploy confirmado com sucesso (build e publicação concluídos no GitHub Actions) — link já está no ar

Versão 4 — Nova identidade visual, câmbio na tela inicial e RFs pendentes do novo levantamento

Data: 15/09/2026

A partir do novo levantamento de requisitos e do protótipo de telas enviados pelo usuário, esta etapa aplicou a nova paleta de cores do app e implementou os requisitos novos/pendentes que já podiam ser feitos sem depender de validação do orientador — ou seja, tudo exceto o assistente de IA (RF33–RF40) e a leitura de notas fiscais por visão computacional (RF41–RF44), que o próprio levantamento pede para validar antes de implementar.

- Nova paleta de cores "Mapa & Bússola" (índigo, verde bússola, dourado, terracota, areia), aplicada trocando só os valores em constants.dart — nenhuma tela precisou ser redesenhada
- RF45/RF46 — conversão de câmbio na tela inicial: cada viagem agora tem uma moeda local (definida ao cadastrar/editar a viagem), e o dashboard mostra o total gasto também convertido para a moeda padrão do usuário, com seletor de moeda e horário da última cotação (RNF23). Conversão via API gratuita open.er-api.com, com tratamento de erro caso o serviço esteja indisponível
- RF14 — seleção manual da viagem ativa exibida no dashboard, via ícone de estrela na lista de viagens (antes era sempre escolhida automaticamente)
- RF29 — seletor de moeda padrão funcional na tela de Perfil (também corrigido um bug em que editar o perfil ou trocar a foto resetava a moeda do usuário de volta para BRL sem querer)
- RF30/RF31/RF32 — novas telas de Configurações, Sobre o app e Suporte, antes eram botões sem função no Perfil
- RF07 — Termos de Uso e Política de Privacidade ganharam tela própria com conteúdo real (LGPD), em vez de um texto estático sem link
- RF22 — anexo de comprovante (foto) às despesas, reaproveitando o Supabase Storage já usado para fotos de perfil/viagem
- RF23 — exportação das despesas filtradas em CSV, gerado no app e disponibilizado por um link do Supabase Storage
- Banco de dados: novas colunas viagens.moeda_local, despesas.foto_url e profiles.viagem_ativa_id — é necessário rodar o script supabase/migracao_v4.sql uma vez no SQL Editor do Supabase para o projeto já existente (o app foi feito para continuar funcionando mesmo antes disso, só sem essas funcionalidades novas)
- RF33–RF40 (assistente de IA) e RF41–RF44 (leitura de nota fiscal) ficam para a próxima etapa, após validação com o orientador — quando avançar, será usada a API da Anthropic (Claude) para as duas frentes
- RF23 exporta em CSV (não em PDF) e o app ainda não foi testado rodando de verdade após esta etapa — recomenda-se testar câmbio, viagem ativa, comprovante e exportação antes da apresentação

Versão 5 — Assistente de IA no app (RF33–RF40)

Data: 16/09/2026

Após validação do orientador, foi implementado o assistente de IA do app: uma tela de chat onde o usuário pode tirar dúvidas sobre a viagem ativa, o orçamento e os gastos, e pedir dicas de economia. O assistente usa a API do Gemini (Google AI Studio) — inicialmente o plano era usar a API da Anthropic (Claude), mas ela exige cartão de crédito já no cadastro (mesmo para o crédito gratuito de teste), então o provedor foi trocado para o Gemini, que fornece uma chave de API gratuita sem exigir cartão. O app Flutter nunca fala diretamente com a API de IA nem guarda a chave — isso passa por uma Edge Function do Supabase, que funciona como um servidor intermediário seguro, independente do provedor escolhido.

- Por que uma Edge Function? A chave da API de IA é um segredo de verdade, diferente da anonKey do Supabase (que é pública por design e protegida por Row Level Security). Se essa chave fosse colocada direto no código do app Flutter, qualquer pessoa poderia extraí-la abrindo o DevTools na versão web — por isso ela fica guardada só no servidor, como 'secret' do projeto Supabase, e nunca é commitada no Git
- Nova Edge Function supabase/functions/chat-ia/index.ts (Deno): recebe a pergunta do usuário e o histórico recente da conversa, monta um prompt de sistema com os dados da viagem ativa (nome, destino, orçamento, total gasto, últimas despesas), chama a API do Gemini (modelo gemini-3.5-flash, trocável por variável de ambiente) e devolve a resposta. Só funciona para usuários autenticados (verificação de sessão já é padrão do Supabase para Edge Functions)
- Banco de dados: novas tabelas conversas e mensagens, com Row Level Security (cada usuário só acessa as próprias conversas/mensagens) — script supabase/migracao_v5.sql para quem já tinha o projeto criado antes desta etapa
- Novo ChatService (lib/services/chat_service.dart): cria/recupera a conversa do usuário, salva as mensagens no banco e aciona a Edge Function
- Nova tela ChatScreen (lib/screens/chat_screen.dart): interface de chat com balões de mensagem, indicador de 'digitando...' e mensagem de boas-vindas explicando o que perguntar
- Novo item 'Assistente' na barra de navegação inferior do app, entre Despesas e Perfil
- Leitura de nota fiscal por visão computacional (RF41–RF44) fica para a próxima etapa, seguindo a ordem combinada com o usuário
- Passo pendente do lado do usuário (fora do código): criar uma chave de API gratuita em aistudio.google.com (não exige cartão), publicar a Edge Function e configurar o secret GEMINI_API_KEY no painel do Supabase — sem isso, o chat fica no ar mas o assistente não consegue responder. Ainda não testado rodando de verdade

Versão 6 — Leitura automática de nota fiscal por visão computacional (RF48-RF51) e novo levantamento de requisitos

Data: 24/09/2026

O usuário enviou um novo levantamento de requisitos e um protótipo atualizado, com uma mudança importante: o 'Agente de IA' deixou de ser um chat simples e virou o 'Agente de vIAgens', um consultor de viagens completo (entrevista prévia, roteiro com gastos estimados, indicação de passagem/hospedagem/restaurante, apoio a viagens de carro e documentação/vacinas — agora RF33 a RF47). Esse bloco inteiro continua exigindo validação com o orientador antes de ser implementado, igual da vez passada, então esta etapa focou no que já podia avançar com segurança: a leitura automática de nota fiscal (agora RF48-RF51) e a confirmação de que a conversão de câmbio já implementada atende ao que virou RF52/RF53.

- RF48/RF49 — ao cadastrar uma despesa nova, o usuário escolhe entre 'Escanear nota' ou 'Digitar manualmente'. Ao escanear (câmera ou galeria), a foto é enviada para uma nova Edge Function que usa o Gemini em modo multimodal (lê imagem e texto) para reconhecer estabelecimento, valor, data e sugerir uma categoria, pré-preenchendo o formulário
- RF50 — os dados reconhecidos aparecem nos mesmos campos editáveis do formulário, com um aviso pedindo para conferir antes de salvar — nada é salvo automaticamente sem revisão
- RF51 — o modo 'Digitar manualmente' continua disponível para quando não há nota fiscal, sem nenhuma mudança no fluxo que já existia
- RNF21 — quando a IA não consegue ler os dados com confiança (foto ruim, não é uma nota fiscal, etc.), a tela avisa e deixa os campos em branco para preenchimento manual, sem travar o cadastro da despesa
- Nova Edge Function supabase/functions/extrair-nota, que reaproveita o MESMO secret GEMINI_API_KEY já configurado para o assistente de IA (v5) — não é necessário cadastrar nenhuma chave nova nem pagar por um segundo serviço
- A foto escaneada da nota fiscal passa a ser usada automaticamente como o comprovante da despesa (RF22, v4) — a nota que o usuário já tirou vira a prova de pagamento, sem precisar anexar de novo
- Documentação: seções de modelos e serviços atualizadas (ResultadoExtracao, NotaFiscalService), e nota explicando a renumeração dos requisitos do Agente de IA (RF33-40 antigo → RF33-47 no levantamento atual) para não gerar confusão na leitura do histórico de versões
- RF33-47 (Agente de vIAgens completo) permanecem pendentes de validação com o orientador — o assistente de chat simples da v5 continua no ar, mas é uma versão inicial, não o Agente de vIAgens final
- Renomeada a tela e os textos do assistente de IA de 'Assistente de IA' para 'Agente de vIAgens' (título da tela, mensagem de boas-vindas e item da barra de navegação inferior, que ficou 'Agente' por limitação de espaço) — o nome já passa a refletir a terminologia do novo levantamento, mesmo a função por trás continuando a ser a versão inicial (chat simples da v5), não o Agente de vIAgens completo (RF33-47), que segue pendente de validação
- Edge Function extrair-nota publicada pelo painel do Supabase — cadastro de despesa por escaneamento de nota fiscal ainda pendente de teste com notas reais
- No teste do assistente de IA (chat), apareceu erro de conexão ('Failed to fetch') ao chamar a Edge Function chat-ia. Causa provável: o código dessa função foi reescrito para usar o Gemini (troca de provedor feita antes desta etapa), mas a versão publicada no Supabase ainda é a anterior — a função precisa ser reimplantada pelo painel (mesmo fluxo usado para a extrair-nota, colando o conteúdo atual de supabase/functions/chat-ia/index.ts) para passar a usar o Gemini de fato

Versão 7 — Leitura de nota fiscal por QR Code (NFC-e), além da foto (RF48-RF51)

Data: 05/10/2026

O usuário perguntou se dava para ler a nota fiscal pelo QR Code em vez de só por foto, e foi claro que a ideia era decodificar o QR de verdade (como um leitor de código de barras), não tirar uma foto do QR e pedir pra IA 'enxergar' o código nela. Pesquisamos como funciona o QR Code de uma NFC-e brasileira: ele não traz o valor nem os itens da compra, só uma URL de consulta no site da Receita do estado emissor, com a chave de acesso de 44 dígitos. Por isso, a leitura virou um fluxo de duas partes — decodificação do QR localmente no aparelho (sem IA, via pacote mobile_scanner) e, depois, uma Edge Function no servidor que busca essa página de consulta e usa o Gemini (em modo texto, não mais visão) para ler os dados ali, o que tende a ser mais confiável que adivinhar a partir de uma foto. Também avaliamos o impacto no tamanho do app antes de implementar: a biblioteca usa APIs nativas (ML Kit/Vision) e soma só ~200 KB a 2,4 MB no Android, e nada no iOS, então não pesa no app.

- Adicionado o pacote mobile_scanner (^7.3.0) ao pubspec.yaml, para decodificar QR Code localmente, pela câmera, em Android/iOS/Web — sem precisar de rede nem de IA nesse passo
- Nova tela EscanearQrcodeScreen (lib/screens/escanear_qrcode_screen.dart): abre a câmera com uma moldura de leitura, detecta o QR Code automaticamente e devolve o conteúdo lido para a tela de despesas
- Nova Edge Function supabase/functions/extrair-nota-qrcode: recebe o conteúdo do QR Code, extrai a chave de acesso de 44 dígitos, identifica o estado emissor pelos 2 primeiros dígitos da chave (tabela de códigos do IBGE) e, se for uma nota da Paraíba (PB), busca a página oficial de consulta da SEFAZ-PB e pede ao Gemini (modo texto) para extrair estabelecimento, valor, data e categoria sugerida — mesmo contrato de resposta (ResultadoExtracao) já usado pela extrair-nota, reaproveitando o mesmo secret GEMINI_API_KEY
- Cobertura inicial limitada à Paraíba (PB), de propósito: os outros 26 estados têm páginas de consulta diferentes entre si, e implementar todas de uma vez seria desproporcional para o escopo do TCC. Para nota de outro estado (ou QR ilegível), a função devolve um erro claro e a tela de despesas cai automaticamente no fluxo por foto/manual já existente — nada trava para o usuário
- NotaFiscalService ganhou o método extrairPorQrCode(qrContent), espelhando o mesmo padrão de segurança do extrair() por foto (a chave de IA nunca sai do servidor)
- DespesasScreen: a lógica que aplica o resultado da IA aos campos do formulário (antes só dentro da função de escanear por foto) foi extraída para aplicarResultado(), reutilizada tanto pelo fluxo de foto quanto pelo novo fluxo de QR Code — evita duplicar a lógica de preenchimento/aviso
- Novo botão 'Ler QR Code da nota (PB)' na tela de nova despesa, abaixo das opções de Câmera/Galeria já existentes
- mobile_scanner ainda não foi testado em um app rodando de verdade neste ambiente (sem Flutter instalado aqui); e a Edge Function extrair-nota-qrcode também precisa ser publicada pelo painel do Supabase (mesmo processo das demais) antes de funcionar de ponta a ponta

Versão 8 — Remoção da leitura automática de nota fiscal e correção do botão "+"

Data: 10/10/2026

Depois de ver a leitura automática de nota fiscal (foto, v6, e QR Code, v7) funcionando de verdade na tela de despesas, o usuário decidiu reverter essa frente: o cadastro de despesa volta a ser sempre manual, com apenas um anexo de foto do comprovante simples e opcional (RF22), sem nenhuma chamada de IA nesse passo. O usuário também reportou, com uma captura de tela, que o botão "+" de nova despesa (um FloatingActionButton no canto inferior direito) estava sobrepondo o valor do Total mostrado na parte inferior da tela de despesas — corrigido nesta etapa, movendo o botão para um ícone discreto no canto superior direito da barra de título.

- DespesasScreen: removidos o alternador "Escanear nota"/"Digitar manualmente", as funções aplicarResultado/escanear/escanearQrCode, os botões de Câmera/Galeria/"Ler QR Code da nota (PB)" e o aviso de dados reconhecidos automaticamente — o formulário de despesa volta a mostrar sempre os mesmos campos manuais (categoria, descrição, valor, data, forma de pagamento)
- O anexo de comprovante (RF22, já existente desde a v4) continua disponível como botão "Anexar/Trocar/Remover" na seção "Comprovante (opcional)" do formulário — sem nenhuma IA envolvida, é só a foto escolhida da galeria/câmera sendo guardada no Supabase Storage
- Removidos os arquivos lib/screens/escanear_qrcode_screen.dart, lib/services/nota_fiscal_service.dart e lib/models/resultado_extracao.dart, e as pastas das Edge Functions supabase/functions/extrair-nota e supabase/functions/extrair-nota-qrcode
- Removida a dependência mobile_scanner (^7.3.0) do pubspec.yaml, já que não há mais leitura de QR Code no app
- Corrigido o botão "+" de nova despesa: antes era um FloatingActionButton (posição padrão do Flutter, canto inferior direito), que sobrepunha a barra de "Total" na parte inferior da tela de despesas (bug relatado pelo usuário com captura de tela). Agora é um IconButton minimalista nas actions da AppBar, no canto superior direito, ao lado do botão de exportar (CSV) — mesmo comportamento de antes (abre o formulário de nova despesa, só aparece quando há uma viagem selecionada)
- Documentação: removidas as seções 2.5 (ResultadoExtracao) e 3.5 (NotaFiscalService) do documento técnico, e atualizada a descrição da DespesasScreen (seção 4) e das pendências (seção 6) para refletir o estado atual do app
- As Edge Functions extrair-nota e extrair-nota-qrcode deixaram de ser chamadas pelo app, mas continuam publicadas no painel do Supabase — removê-las do repositório não as remove de lá; quem quiser liberar esse espaço precisa excluí-las manualmente pelo painel do Supabase
- Ainda não testado rodando de verdade neste ambiente (sem Flutter instalado aqui) — recomenda-se rodar 'flutter pub get' (para atualizar o lockfile após remover o mobile_scanner) e conferir o cadastro/edição de despesa e a nova posição do botão "+" antes da apresentação

Versão 9 — Parte do Agente de vIAgens completo: roteiro, passagem/hospedagem/restaurante e apoio a carro (RF33-RF47)

Data: 10/10/2026

O usuário perguntou o que o Agente de vIAgens (tela de chat) realmente consegue fazer hoje, e a resposta deixou claro que era só um chat de perguntas/respostas sobre a viagem ativa — nada do planejamento mais completo (roteiro, passagem/hospedagem/restaurante, apoio a carro, documentação/vacinas) que o nome sugere e que o levantamento de requisitos descreve como RF33-RF47. Ao ser perguntado se dava para adicionar essa parte, o usuário escolheu avançar com roteiro, indicação de passagem/hospedagem/restaurante e apoio a viagens de carro (deixando documentação/vacinas de fora por enquanto), e decidiu que as sugestões de passagem/hospedagem/restaurante devem ser só estimativas da própria IA, sem contratar nenhuma API paga de viagens — mais simples e gratuito para o escopo do TCC, ainda que isso signifique preços/opções aproximados, não reais.

- Novo botão "Planejar viagem com IA" na tela do Agente de vIAgens, logo abaixo da barra de título: abre um formulário de entrevista (destino, data de início/fim, orçamento aproximado, meio de transporte, preferências), pré-preenchido com os dados da viagem ativa quando disponíveis
- Edge Function chat-ia ganhou um segundo modo, 'roteiro' (o modo padrão, sem esse campo, continua sendo o chat de sempre) — mesma função, mesmo secret GEMINI_API_KEY, só muda o prompt enviado ao Gemini
- No modo roteiro, o prompt pede um roteiro dia a dia, estimativa de custos por categoria (comparando com o orçamento informado), sugestões de passagem, de hospedagem e de restaurantes (faixas de preço aproximadas) e, quando o meio de transporte informado é carro, dicas de estrada (trecho, pedágio estimado, pontos de parada, documentação do veículo)
- O prompt instrui explicitamente o Gemini a nunca inventar nomes reais de companhias aéreas, hotéis ou restaurantes (só tipos/categorias) e a deixar claro que os valores são estimativas, sem consulta a preços reais — não há nenhuma integração com API de passagem/hospedagem paga
- ChatService ganhou os métodos resumirEntrevista(...) (monta o texto da entrevista salvo como a 'pergunta' do usuário no histórico) e planejarViagem(...) (aciona o modo roteiro e salva a resposta) — reaproveita a mesma tabela de mensagens e a mesma persistência do chat comum, sem nenhuma tabela nova no banco
- Documentação/vistos/vacinas (também parte do RF33-RF47) ficou de fora desta etapa, por escolha do usuário, e segue pendente de validação com o orientador junto com o resto do Agente de vIAgens completo
- Documentação: seção 3.4 (ChatService) atualizada com o novo modo e método, descrição da ChatScreen (seção 4) atualizada, nota sobre a renumeração do Agente de IA (seção 6) e pendências atualizadas para refletir o que já foi implementado nesta etapa
- Ainda não testado rodando de verdade neste ambiente (sem Flutter instalado aqui) — recomenda-se testar o botão 'Planejar viagem com IA' com destinos reais antes da apresentação, inclusive o caminho de meio de transporte 'Carro' (que deve trazer a seção extra de dicas de estrada)

Versão 10 — Correção da exportação de despesas (RF23) — bug de codificação e erro genérico

Data: 10/10/2026

O usuário perguntou para que serve a exportação de despesas e relatou que ela estava dando erro. Lendo o código, encontrei um bug real: o CSV era convertido para bytes usando codeUnits (código UTF-16 de cada caractere) em vez de UTF-8 de verdade, o que corrompe qualquer caractere fora do alfabeto latino básico — por exemplo, um emoji digitado na descrição de uma despesa — na hora de enviar o arquivo para o Supabase Storage. Além disso, o aviso de erro mostrado na tela era sempre o mesmo texto genérico ("Erro ao gerar exportação"), sem nenhum detalhe do motivo real, então não dava para saber se o problema era esse ou outra coisa (ex.: bucket/política de Storage no Supabase) só olhando o app.

- DespesasScreen._exportar(): o CSV agora é convertido para bytes com utf8.encode(csv) (import 'dart:convert') em vez de csv.codeUnits, evitando corromper caracteres fora do Latin-1 básico
- Adicionado um BOM (marca de ordem de bytes, 0xEF 0xBB 0xBF) no início do arquivo CSV, para o Excel no Windows reconhecer a codificação UTF-8 e exibir os acentos corretamente ao abrir o arquivo exportado
- A mensagem de erro de exportação deixou de ser um texto fixo ("Erro ao gerar exportação") e agora mostra o motivo real devolvido pela exceção (ex.: erro do Supabase Storage), além de logar o erro completo no console, para facilitar o diagnóstico de qualquer problema remanescente
- O que a função faz, para registro: gera um CSV (Data, Descrição, Categoria, Forma de pagamento, Valor e uma linha de Total) a partir das despesas filtradas na tela, envia esse arquivo para o Supabase Storage e mostra um link para copiar/baixar — pensado para o caso de uso de 'prestação de contas' que dá nome ao app
- Documentação: seção 6 (pendências) atualizada — a exportação de despesas deixou de ser uma das telas com tratamento de erro genérico
- Ainda não testado rodando de verdade neste ambiente (sem Flutter instalado aqui) — recomenda-se testar a exportação com uma despesa cuja descrição tenha emoji ou acentos antes da apresentação, e, se o erro genérico ainda aparecer (agora com detalhe), investigar a causa real (ex.: bucket/política 'fotos' no Supabase) a partir da mensagem

Versão 11 — Link de acesso do app mais curto — repositório renomeado no GitHub Pages

Data: 11/10/2026

O usuário pediu um link de acesso mais bonito que o padrão do GitHub Pages (https://viajaaibolso-design.github.io/viaja-ai-bolso/). Avaliamos três caminhos: domínio próprio registrado (pago ou via GitHub Student Pack), deploy em outro host gratuito com subdomínio escolhido (Netlify/Vercel/Cloudflare Pages), ou encurtar o link atual renomeando o repositório no GitHub. O usuário escolheu a opção gratuita e mais rápida: renomear o repositório para exatamente 'viajaaibolso-design.github.io' (nome da organização), que faz o GitHub Pages passar a servir o site na raiz do domínio, sem o sufixo '/viaja-ai-bolso/'. O usuário fez a renomeação pelo site do GitHub (ação exclusiva de dono/admin do repositório). Também foi avaliada e descartada a opção de renomear a própria organização do GitHub (que removeria o 'aa' duplicado do nome): é uma ação de maior risco (nome antigo fica livre para registro por terceiros, pacotes/imagens mudam de endereço, sem garantia de redirecionamento do link do GitHub Pages) para corrigir apenas uma letra, então não foi feita.

- Repositório no GitHub renomeado de 'viaja-ai-bolso' para 'viajaaibolso-design.github.io' (ação feita pelo usuário no site do GitHub).
- Link de acesso ao app mudou de https://viajaaibolso-design.github.io/viaja-ai-bolso/ para https://viajaaibolso-design.github.io/.
- Ajustado .github/workflows/deploy.yml: o parâmetro --base-href do build Flutter Web mudou de '/viaja-ai-bolso/' para '/', para que os caminhos dos arquivos estáticos (JS, CSS, assets) do app continuem corretos no novo endereço raiz.
- Sem mudança de código Dart, banco de dados ou Edge Functions — mudança restrita à infraestrutura de deploy (CI/CD do GitHub Actions).

Versão 12 — Dashboard — toque no indicador de percentual revela o saldo real restante

Data: 20/10/2026

O usuário pediu uma forma rápida de ver, na tela inicial, quanto dinheiro ainda resta do orçamento da viagem, sem precisar abrir a lista de despesas. O indicador circular de percentual gasto (card 'Resumo da sua viagem') passou a ser tocável: ao tocar nele, o texto no centro do círculo alterna entre o percentual gasto ('X%') e o saldo real restante, calculado como orçamento menos total gasto e formatado na moeda local da viagem (mesma função formatarMoeda já usada no resto do app). Um rótulo pequeno abaixo do círculo ('toque p/ saldo' / 'toque p/ %') indica a interação disponível, já que não havia nenhum indicador visual de que o elemento era clicável antes dessa mudança.

- lib/screens/dashboard_screen.dart: indicador circular de percentual envolvido por um GestureDetector; estado _mostrarSaldoReal (bool) adicionado a _DashboardScreenState para controlar o que é exibido.
- Texto central do indicador agora alterna, a cada toque, entre '${percentual}%' e o saldo restante (orçamento - total_gasto) formatado com formatarMoeda(), dentro de um FittedBox para evitar que o valor em moeda (mais longo que o percentual) estoure o espaço do círculo de 70x70.
- Rótulo de apoio 'toque p/ saldo' / 'toque p/ %' adicionado abaixo do círculo, em fonte pequena (9px, branco translúcido), só para indicar a interação — não altera o layout geral do card.
- Sem mudança de banco de dados, Edge Functions ou outras telas — alteração restrita à DashboardScreen (apenas UI/estado local).

Versão 13 — Ícones de categoria incompletos + otimização do carregamento do dashboard

Data: 29/10/2026

Duas melhorias pedidas pelo usuário após a apresentação do pré-projeto. Primeiro, um bug visual: a tabela categorias no banco tem 7 categorias (Alimentação, Transporte, Hospedagem, Lazer, Compras, Saúde, Outros), mas a função que traduz o nome do ícone salvo no banco para um ícone do Flutter só reconhecia 4 delas — Hospedagem, Compras e Saúde caíam no ícone genérico de reticências (more_horiz) tanto na lista de despesas quanto no resumo do dashboard, em vez de mostrar um ícone correspondente à categoria. Segundo, uma otimização de desempenho: o usuário notou que, ao cadastrar uma viagem nova e voltar para a tela inicial, o carregamento demorava muito — a causa era o DashboardService.getDashboard() fazer 5 consultas ao banco uma atrás da outra (sequenciais), sendo que nenhuma delas depende do resultado das outras.

- lib/screens/despesas_screen.dart e lib/screens/dashboard_screen.dart: função _iconeCategoria() ganhou os casos 'hotel' (Icons.hotel), 'shopping_bag' (Icons.shopping_bag), 'medical_services' (Icons.medical_services) e 'category' (Icons.category), cobrindo agora as 7 categorias cadastradas no banco (antes só 4 tinham ícone próprio).
- lib/services/viagem_service.dart: DashboardService.getDashboard() reescrito para disparar as 5 consultas (perfil/viagem ativa, resumo das viagens, gastos de hoje, maior gasto, últimas despesas) de uma vez com Future.wait, em vez de uma await atrás da outra — o tempo de carregamento passa a ser o da consulta mais lenta, não a soma de todas. A tolerância a falha da consulta de perfil (caso a migração v4 ainda não tenha sido aplicada no banco) foi preservada, isolada numa função interna com try/catch.
- Sem mudança de schema do banco, Edge Functions ou comportamento visível além da correção dos ícones — o resultado retornado pelo dashboard continua no mesmo formato de antes.

Versão 14 — Agente de vIAgens — roteiro ficava carregando para sempre sem erro

Data: 30/10/2026

O usuário reportou que, ao pedir pra IA gerar um roteiro de viagem (entrevista de planejamento), a tela ficava carregando indefinidamente, sem nunca mostrar erro nem resposta. Investigamos o fluxo completo (ChatService -> Edge Function chat-ia -> API do Gemini) e descartamos a hipótese de nome de modelo inválido (confirmado por pesquisa: o modelo configurado, gemini-3.5-flash, não foi desativado pela Google, só passou a redirecionar automaticamente para gemini-3.6-flash). A causa real é que nenhuma chamada à Edge Function tinha um limite de tempo no lado do app: se o servidor demorasse mais que o normal (cold start da function, lentidão pontual da API do Gemini, etc.), o Flutter ficava esperando a resposta sem nenhum prazo, deixando o indicador de carregamento preso na tela em vez de mostrar um erro que o usuário pudesse entender e tentar de novo.

- lib/services/chat_service.dart: ChatService.enviarMensagem() (chat comum) e ChatService.planejarViagem() (roteiro) agora usam .timeout(...) na chamada à Edge Function chat-ia — 45s para o chat comum, 60s para o roteiro (que pede uma resposta mais longa da IA). Ao estourar o prazo, lança uma mensagem de erro específica em vez de deixar a Future pendente para sempre; a tela de chat já tinha tratamento de erro (SnackBar) pronto pra esse caso, então nenhuma mudança foi necessária em chat_screen.dart.
- Nenhuma mudança na Edge Function chat-ia nem no modelo de IA configurado (GEMINI_MODEL) — o problema era só a falta de prazo no lado do cliente. Se a demora persistir mesmo com o timeout mostrando o erro, o próximo passo é olhar os logs da Edge Function no painel do Supabase para ver se é lentidão pontual da API do Gemini ou algo recorrente.